

ASSIGNMENT 9

Q1: What is Behavior Driven Development (BDD)? Explain the role of Cucumber in BDD. Write a simple example of a Gherkin feature file with Given-When-Then.

Behavior Driven Development (BDD):

BDD is a software development approach where developers, testers, and business stakeholders collaborate using simple, natural language to describe the application's behavior before development.

Role of Cucumber in BDD

- Cucumber is a BDD testing tool.
- It uses Gherkin language (Feature, Scenario, Given, When, Then).
- It converts feature files into executable automated tests.
- It improves communication between technical and non-technical team members.
- It supports Selenium for browser automation.

Example Gherkin Feature File:

Feature: Login Functionality

Scenario: Successful Login

Given User is on the Login page

When User enters valid username and password

And User clicks on Login button

Then User should be redirected to the Home page

Q2: You are testing a user login functionality with Cucumber.

Write a Feature file for login.

Write the corresponding Step Definition (Java) for at least one step.

Feature File:

Feature: User Login

Scenario: Valid Login

Given User opens the login page

When User enters username "student"

And User enters password "Password123"

And User clicks the submit button

Then User should be redirected to the logged In successfully page

Step Definition:

```
package Stepdefinition;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import io.cucumber.java.en.And;
import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;

public class Login {

    WebDriver driver;

    @Given("User opens the login page")
    public void user_opens_the_login_page() {

        // Write code here that turns the phrase above into concrete actions
    }
}
```

```

        driver = new ChromeDriver();

        driver.manage().window().maximize();

driver.get("https://practicetestautomation.com/practice-test-login/");

    }

    @When("User enters username {string}")
    public void user_enters_username(String string) {

        // Write code here that turns the phrase above into concrete actions

        driver.findElement(By.id("username")).sendKeys("student");

    }

    @And("User enters password {string}")
    public void user_enters_password(String string) {

        // Write code here that turns the phrase above into concrete actions

        driver.findElement(By.id("password")).sendKeys("Password123");

    }

    @And("User clicks the submit button")
    public void user_clicks_the_submit_button() {

        // Write code here that turns the phrase above into concrete actions

        driver.findElement(By.id("submit")).click();

    }

    @Then("User should be redirected to the logged In successfully page")
    public void user_should_be_redirected_to_the_logged_in_successfully_page() {

        // Write code here that turns the phrase above into concrete actions

        System.out.println("successfull login");

    }

}

```

Q3: Explain the advantages of using Cucumber with Selenium. How do Data Tables, Background, and Hooks improve test reusability? Provide examples.

Advantages of Cucumber with Selenium

- Easy to understand using Gherkin language.
- Supports BDD approach.
- Improves collaboration among developers, testers, and business users.
- Reusable test scenarios.
- Easy maintenance of test scripts.

Data Tables, Background, and Hooks are Cucumber features that help **improve test reusability, readability, and maintainability** by reducing duplicate code and making test scenarios easier to manage.

1. Data Tables

- Used to pass multiple sets of test data to a single step definition. Instead of writing separate scenarios for different inputs, can provide the data in a tabular format.

Example:

▪ Feature File:

Feature: Login functionality using datatable

Scenario: Login with multiple users

Given User is on the login page of the website

When User enters username and password

	username		password	
	standard_user		secret_sauce	
	problem_user		secret_sauce	

▪ **Step Definition Class:**

`@Given("User is on the login page of the website")`

```
public void user_is_on_the_login_page_of_the_website() {  
    // Write code here that turns the phrase above into concrete actions
```

```
        driver = new ChromeDriver();  
        driver.manage().window().maximize();
```

```
    driver.get("https://www.saucedemo.com/");
```

```
}
```

`@When("User enters username and password")`

```
public void user_enters_username_and_password(io.cucumber.datatable.DataTable  
dataTable) {
```

```
    // Write code here that turns the phrase above into concrete actions
```

```
        List<Map<String, String>> users = dataTable.asMaps(String.class,  
String.class);
```

```
        for (Map<String, String> user : users) {
```

```
            driver.findElement(By.id("user-  
name")).sendKeys(user.get("username"));
```

```
            driver.findElement(By.id("password")).sendKeys(user.get("password"));
```

```
            driver.findElement(By.id("login-button")).click();
```

2. Background

- Used to define common steps that are executed before every scenario in a feature file.

Example:

Feature: Login with background

Background: user login

Given User is on the login page

Scenario: Login with valid credentials

When User enters "standard_user" and "secret_sauce"

And Click on login button

Then User should be redirected to the products page

Scenario: Login with locked user

When User enters "locked_out_user" and "secret_sauce"

And Click on login button

Then User should see an error message

3. Hooks

- Hooks are special methods used for setup and cleanup before or after scenarios without being mentioned in the feature file.

Common Hooks:

- @Before
- @After
- @BeforeStep
- @AfterStep

Example:

@Before

```
public void setup() {  
    System.out.println("Browser Opened");  
}
```

@After

```
public void teardown() {  
    System.out.println("Browser Closed");  
}
```

Q4: What is GitHub? Explain how version control helps a testing/automation project. Write the Git commands for the following:

Clone a repository

Create a new branch

Commit changes

Push changes to remote

GitHub is a cloud-based platform used to store Git repositories and collaborate on software development.

Benefits of Version Control

- Tracks code changes.
- Supports teamwork.
- Maintains code history.
- Allows rollback to previous versions.
- Manages multiple branches.

Git Commands:

1. Clone a Repository

git clone <https://github.com/username/repository.git>

2. Create a New Branch

git checkout -b feature_branch

3. Commit Changes

git add .

git commit -m "Added login automation"

4. Push Changes to Remote

git push origin feature_branch

Q5: Explain the steps to configure Jenkins with Maven for running Selenium scripts. Include:

Creating a Maven job

Adding dependencies in pom.xml

Scheduling the job

1. Create a Selenium Maven Project

1. Open Eclipse IDE.
2. Select File → New → Maven Project.
3. Click Next.
4. Choose Create a Simple Project (Skip Archetype Selection).
5. Enter the Group Id and Artifact Id details
6. Click Finish.

7. Create Selenium test classes and a testng.xml file in the project.

2. Add Dependencies in pom.xml

Example:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.testing</groupId>

  <artifactId>SeleniumMavenDemo</artifactId>

  <version>0.0.1-SNAPSHOT</version>


  <dependencies>

    <!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.45.0</version>
      <scope>compile</scope>
    </dependency>

    <!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.12.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

</project>
```

3. Push the Project to GitHub

1. Create a new repository on GitHub.
2. Initialize Git in the Eclipse project.
3. Commit the project files.
4. Push the project to the GitHub repository.
5. Copy the repository URL (HTTPS), as it will be used in Jenkins.

4. Create a Maven Job

1. From the Jenkins Dashboard, click New Item.
2. Enter a job name (e.g., SeleniumJob).
3. Select Maven Project (before that Maven should be configured in Jenkins)
4. Click Ok.
5. Under Source Code Management, select Git.
6. Enter the GitHub repository URL.
7. Specify the branch to build (e.g., */main).

5. Scheduling the Job

1. Open the Jenkins job.
2. Click Configure.
3. Select Build periodically.
4. Enter a Cron expression.

6. To Run the Job

1. Open the Jenkins job.
2. Click Build Now.
3. Wait for the build to complete.
4. Click the build number and open Console Output to view the execution log.
 - If all tests pass, Jenkins displays: BUILD SUCCESS
 - If any test fails, Jenkins displays: BUILD FAILURE